

SentinelCleanAIMT

Slide 1: Title & Overview

- **Project Name:** SentinelCleanAIMT: Autonomous Edge Vision Framework for Intelligent Robotic Maintenance
- **Team Member Name:** Cuong Dang (Individual Project)
- **Project Tier:** Tier 2 (Advanced - Custom Fine-Tuning & Local Robotics Orchestration)

Slide 2: The Problem

- **Real-World Problem:** Current commercial cleaning robots operate blindly using raw proximity sensors. They cannot distinguish between dry debris (vacuumable) and liquid spills (which requires halting the vacuum mechanism to prevent destroying internal machinery filters).
- **Who Cares:** Smart home manufacturers, industrial janitorial automation companies, and logistics warehouse operators.
- **Why It Matters:** Traditional vacuuming over wet spills ruin multi-thousand-dollar floor-cleaning units, destroys internal filtration systems, and spreads stains across clean surfaces, causing severe structural property damage.

Slide 3: Your Solution

- **One-Sentence Description:** An intelligent, privacy-first floor maintenance system that uses vision-based edge computing and cooperative telemetry to detect and classify multi-state hazards.
- **Workflow Diagram:**
 [Input Imagery (Camera)] → [Local RAM Preprocessing] → [YOLOv11 Edge Inference & V2X Fusion] → [ROS2 Behavior Tree Logic] → [Motor Control / Output Telemetry]

Slide 4: Technical Approach

- **CV Technique:** Real-time Object Detection and Class-Agnostic Geometric Prompting.
- **Model Architecture:** CNN + Vision Transformer (ViT) Hybrid Core.
- **Model & Usage:** Ultralytics YOLOv11 (Fine-tuned via Transfer Learning) paired with Meta SAM 2 (Pretrained Base Weights for Zero-Shot boundary tracking).
- **Framework:** PyTorch, Ultralytics SDK, and local ROS2 node structures.
- **Choice Justification:** YOLOv11 delivers high-inference frame rates directly within constrained local RAM to enforce data minimization, while SAM 2 provides perfect boundary maps for oddly shaped liquid puddles without relying on heavy cloud APIs.

Slide 5: Data Plan

- **Source:** 100% Publicly available benchmark datasets sourced via Roboflow Universe and Kaggle Datasets (indoor debris and fluid marking repositories), integrated with simulated digital Metadata payload streams from the OmniID V2X framework.

- **Size:** Open-source image profiles expanded via targeted programmatic geometric and exposure data augmentations inside the PyTorch pipeline.
- **Labels:** dry_debris, liquid_spill, charging_dock, and pet_obstacle.
- **Preparation:** Manual bounding-box annotation validation using Roboflow, enforcing strict Data-Centric AI curation to eliminate ground-shadow illumination artifacts before training.

Slide 6: Success Metrics

- **Primary Metric:** We will measure Mean Average Precision (≥ 0.45 mAP₅₀₋₉₅) and Recall ($\geq 90\%$) for the liquid_spill class to guarantee the robot never accidentally runs over a fluid hazard.
- **Secondary Metric:** We will measure real-time local inference execution latency and expect a target speed $\leq 45\text{ms}$ per image frame to guarantee physical collision avoidance safety margins.

Slide 7: Milestone Plan (10-Week Summer Term)

- **Week 5 (Tonight):** Project Scoping & Architecture Approval. *Milestone: Blueprint Submitted & Initial Repository Established.*
- **Week 6:** End-to-End Pipeline Infrastructure Validation. *Milestone: Pre-trained Base Inferences Functional on 5 Test Frames.*
- **Weeks 7 to 8:** Transfer Learning Integration & ROS2 Orchestration. *Milestone: Custom Weights Trained & Local Actuation Logic Unified.*

- **Week 9:** Model Diagnostic Evaluation & System Optimization. *Milestone: Metric Matrix Recorded & Edge Failure Logs Cataloged.*
- **Week 10:** Production-Grade Packaging & Final Review. *Milestone: System Repository Locked & Video Demonstration Compiled.*

Slide 8: Risks and Resources

- **Top Risks & Plan B:**
 1. *Dual-model inference overhead causes processing latency to exceed the 45ms threshold:* Lock the tracking pipeline to an optimal confidence threshold of 0.25 and run SAM 2 conditionally only when liquid-spill confidence keys trigger.
 2. *Hardware resource constraints trigger local RAM memory overflow during multi-component execution:* Downsample input frame resolution scales or switch the training/testing phase entirely to a **Kaggle Notebook** environment to utilize its longer session limits.
 - **Compute Plan:** Free Google Colab GPU runtime instances for model fine-tuning; local machine architecture for system synthesis.
 - **Estimated Cost: \$0.00.** Completely built using open-source packages and local edge compute paradigms.
-